Mini Course on AI Agent

If you're building SpaceLinear, you don't need to learn all of AI first. Learn **Agentic AI from a builder's perspective**.

Module 1: What is an Agent?

Most beginners think:

```
User → LLM → Response
```

Example:

```
User: Explain photosynthesis
GPT: Explanation...
```

That's just an LLM.

An **agent** can take actions:

```
User
 ↓
LLM
 ↓
Choose Tool
 ↓
Execute Tool
 ↓
Return Result
```

Example:

```
User: Create a note called Biology

Agent:
→ Calls createNote()
```

```
→ Saves note
→ Responds
```

The difference:

- LLM = thinks
- Agent = thinks + acts

Module 2: What are Tools?

Tools are functions the AI can use.

Example:

```
createNote(title, content)

createTask(title, dueDate)

getUpcomingExams()

searchNotes(query)
```

The AI never directly modifies your database.

Instead:

```
AI decides
↓
Tool executes
↓
Database changes
```

Think of tools as the AI's hands.

Module 3: Agent Loop

Most agents follow:

```
Observe
↓
Think
↓
Act
↓
Observe Again
```

Example:

```
User:
Plan my study schedule
```

Agent:

```
1. Get exams
2. Get available time
3. Create schedule
4. Save tasks
5. Return plan
```

This is called an **agent loop**.

Module 4: Memory

Without memory:

```
User:
My exam is in 2 weeks

Tomorrow:

User:
What should I study?

AI:
I don't know.
```

With memory:

```
Store:
Exam = Physics
Date = June 20
```

Now:

```
User:
What should I study?

AI:
Physics revision today.
```

For SpaceLinear:

Short-term memory

Current conversation.

```
Chat history
```

Long-term memory

Database.

```
Notes
Tasks
Flashcards
Study sessions
Exams
```

Module 5: Retrieval (RAG)

Suppose user has:

```
500 notes
```

Sending all notes to the AI is expensive.

Instead:

```
User asks question
↓
Search notes
↓
Get relevant notes
↓
Send only those notes
↓
Generate answer
```

This is Retrieval-Augmented Generation (RAG).

For SpaceLinear:

```
Question:
Explain Newton's Second Law

↓

Search notes

↓

Top 5 relevant notes

↓

Generate answer
```

Module 6: Single-Agent Architecture

Start here.

```
User
 ↓
AI Agent
 ↓
Tools
```

```
 ↓
Database
```

Example:

```
SpaceLinear Agent
├─ Create Note
├─ Edit Note
├─ Create Task
├─ Schedule Study
└─ Generate Quiz
```

One agent controls everything.

Good for MVP.

Module 7: Multi-Agent Architecture

Later:

```
Orchestrator
│
├── Note Agent
├── Planner Agent
├── Quiz Agent
├── Calendar Agent
└── Revision Agent
```

Example:

```
User:
Prepare me for finals
```

Orchestrator:

```
Ask Planner Agent
↓
Ask Revision Agent
↓
```

```
Ask Calendar Agent
↓
Combine results
```

Most startups don't need this initially.

Module 8: Planning

Basic AI:

```
User:
Prepare for exam
```

AI responds immediately.

Agentic AI:

```
Goal:
Prepare for exam

Step 1:
Get exam date

Step 2:
Calculate study days

Step 3:
Generate schedule

Step 4:
Create tasks

Step 5:
Return result
```

The AI creates a plan before acting.

Module 9: State

State is information the workflow carries.

Example:

```json
{
  "exam": "Physics",
  "daysRemaining": 12,
  "chaptersLeft": 5
}
```

Every step updates state.

This is why frameworks like LangGraph are useful.

Module 10: Agent Workflow for SpaceLinear

Imagine:

```
User:
I have Calculus exam in 14 days.
Make a study plan.
```

Workflow:

```
1. Extract exam info

2. Get user availability

3. Check completed chapters

4. Calculate revision load

5. Generate schedule

6. Create calendar sessions

7. Save tasks

8. Return plan
```

This is a real agentic workflow.

Module 11: Common Beginner Mistake

People build:

```
AI Chat
```

and call it an agent.

Reality:

```
AI
+
Tools
+
Memory
+
Workflow
+
State
```

= Agentic System

Module 12: What to Learn Next

Week 1:

- Function/tool calling
- OpenAI API
- Vercel AI SDK

Week 2:

- Supabase
- Database design
- User memory

Week 3:

- RAG
- Embeddings
- Vector search

Week 4:

- LangGraph
- Agent workflows
- State machines

SpaceLinear MVP Architecture

```
Next.js
│
├── Vercel AI SDK
│
├── GPT-5.5
│
├── Supabase
│
├── Study Planner Tools
│
├── Notes Tools
│
├── Task Tools
│
└── Single Agent
```

Build this first.

Do not start with LangGraph, multiple agents, MCP, memory systems, and autonomous planning all at once. A single tool-calling agent that can create notes, schedule study sessions, and generate quizzes will teach you 80% of what agentic AI development actually feels like.